MODUL 08 · BAB III · DIVIDE AND CONQUER DAN RECURRENCE

Paradigma Divide and Conquer dan Merge Sort

Memecah masalah menjadi submasalah, menyelesaikannya secara rekursif, lalu menggabungkan hasilnya.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Menjelaskan tiga tahap paradigma Divide and Conquer: divide, conquer, combine.
- Menjelaskan cara kerja MERGE_SORT dan prosedur MERGE baris demi baris.
- Menelusuri Merge Sort secara lengkap, termasuk urutan pemanggilan rekursif dan proses penggabungan.
- Menurunkan bentuk recurrence $T(n) = 2T(n/2) + \Theta(n)$ dan menjelaskan secara intuitif mengapa hasilnya $\Theta(n \lg n)$.

PRASYARAT

- Modul 02: rekursi dan call stack.
- Modul 04: Insertion Sort (sebagai pembanding).
- Modul 06: notasi Θ.

BAB / MATERI ASAL

Materi 6. Algoritma Divide and Conquer

POSISI DALAM URUTAN BELAJAR

Paradigma perancangan pertama. Recurrence yang dihasilkan diselesaikan secara formal di Modul 09 (Metode Master).

DAFTAR ISI

1. Membagi Masalah Besar menjadi Masalah Kecil 1
2. Paradigma Divide and Conquer 2
3. Algoritma Merge Sort 2
4. Penelusuran Merge Sort 4
5. Analisis Running Time: Lahirnya Recurrence 7
6. Merge Sort dibandingkan dengan Insertion Sort 8
7. Kesimpulan 9

◆ Kuis Pemahaman 10
◆ Kunci Jawaban dan Pembahasan 11
◆ Referensi 12

PENDAHULUAN

1. Membagi Masalah Besar menjadi Masalah Kecil

Insertion Sort (Modul 04) bekerja secara **inkremental**: ia memperbesar bagian yang sudah terurut satu elemen demi satu elemen. Pendekatan ini sederhana, tetapi kasus terburuknya $\Theta(n^2)$. Untuk satu juta data, itu berarti sekitar 10^{12} langkah.

Ada cara berpikir lain yang sering jauh lebih efisien: **pecah** masalah menjadi beberapa masalah yang lebih kecil dengan bentuk yang sama, **selesaikan** masing-masing secara rekursif, lalu **gabungkan** hasilnya.

Paradigma ini disebut **Divide and Conquer** (bagi dan taklukkan). Kita sebenarnya sudah melihat bentuk sederhananya pada Binary Search (Modul 02), yang membuang separuh array pada setiap langkah.

Modul ini membahas paradigma Divide and Conquer melalui contoh klasiknya, **Merge Sort**. Kita akan menelusuri algoritmanya secara lengkap, termasuk latihan dari bahan kuliah yang belum diselesaikan, lalu menurunkan running time-nya dalam bentuk **recurrence**. Penyelesaian recurrence tersebut secara formal menjadi pokok bahasan Modul 09.

PEMBAHASAN INTI

2. Paradigma Divide and Conquer

DEFINISI

Algoritma **Divide and Conquer** menyelesaikan masalah dalam tiga tahap pada setiap tingkat rekursi:

1. **Divide**: membagi masalah menjadi beberapa submasalah yang lebih kecil, berbentuk sama dengan masalah semula.
2. **Conquer**: menyelesaikan masing-masing submasalah secara rekursif. Jika submasalah sudah cukup kecil, selesaikan secara langsung (kasus dasar).
3. **Combine**: menggabungkan solusi submasalah sehingga membentuk solusi masalah semula.

Kunci keberhasilan paradigma ini ada pada dua hal: submasalahnya harus benar-benar lebih kecil (agar rekursi berhenti), dan langkah *combine* harus murah. Jika menggabungkan solusi ternyata semahal menyelesaikan masalah dari awal, membagi masalah tidak memberi keuntungan.

Pada masalah pengurutan n bilangan di array A berukuran $1 \times n$, ketiga tahap itu menjadi:

Tahap	Pada Merge Sort	Biaya
Divide	Membagi array A yang berisi n bilangan menjadi 2 subarray, masing-masing sekitar $n/2$ bilangan, dengan menghitung indeks tengah.	Konstan
Conquer	Mengurutkan masing-masing subarray secara rekursif dengan Merge Sort.	Dua kali biaya mengurutkan $n/2$ bilangan
Combine	Menggabungkan dua subarray yang sudah terurut menjadi satu array terurut dengan prosedur MERGE.	Sebanding dengan n

Tabel 1. Tiga tahap Divide and Conquer pada Merge Sort.

PEMBAHASAN INTI

3. Algoritma Merge Sort

3.1 Prosedur utama MERGE_SORT

MERGE_SORT(A, p, r) // p: indeks pertama, r: indeks terakhir subarray

```
1  if p < r                      // subarray berisi lebih dari satu elemen
2      q = ⌊(p + r) / 2⌋         // divide: indeks tengah, dibulatkan ke bawah
3      MERGE_SORT(A, p, q)       // conquer: urutkan A[p..q]
4      MERGE_SORT(A, q + 1, r)   // conquer: urutkan A[q+1..r]
5      MERGE(A, p, q, r)         // combine: gabungkan dua bagian terurut
```

Pemanggilan awal untuk mengurutkan seluruh array adalah `MERGE_SORT(A, 1, n)`. Penjelasan setiap baris:

- **Baris 1**: jika $p < r$, subarray $A[p..r]$ berisi minimal dua elemen dan masih perlu diurutkan. Jika $p \geq r$, subarray berisi paling banyak satu elemen, yang dengan sendirinya sudah terurut. Inilah **kasus dasar**: fungsi langsung selesai tanpa melakukan apa pun.
- **Baris 2**: menghitung indeks tengah q dengan pembulatan ke bawah, misalnya $\lfloor 7/2 \rfloor = 3$.
- **Baris 3–4**: fungsi memanggil dirinya sendiri untuk bagian kiri $A[p..q]$ dan bagian kanan $A[q+1..r]$. Setelah kedua pemanggilan selesai, masing-masing bagian sudah terurut dari kecil ke besar.
- **Baris 5**: MERGE menggabungkan dua bagian terurut itu menjadi satu subarray $A[p..r]$ yang terurut.

CATATAN KOREKSI TERHADAP SUMBER

Bahan kuliah menulis: "Langkah 3 dan 4 masing-masing akan berhenti apabila nilai $p < r$." Seharusnya rekursi berhenti apabila kondisi $p < r$ bernilai **salah**, yaitu ketika $p \geq r$ (subarray berisi satu elemen). Hal ini juga terlihat pada penelusuran di bahan kuliah sendiri: "`MERGE_SORT(A,1,1)` : if $p < r$ False sebab 1 tidak kecil dari 1", lalu kembali ke fungsi pemanggil.

3.2 Prosedur MERGE

Bahan kuliah merujuk pseudocode MERGE pada buku Cormen dkk. (2009, Bab 2.3.1). Gagasannya seperti menggabungkan dua tumpukan kartu yang masing-masing sudah terurut dengan kartu terkecil di atas: bandingkan dua kartu teratas, ambil yang lebih kecil, letakkan di tumpukan hasil, ulangi sampai habis.

MERGE(A, p, q, r) // A[p..q] dan A[q+1..r] sudah terurut

```
1  n1 = q - p + 1                    // banyaknya elemen bagian kiri
2  n2 = r - q                        // banyaknya elemen bagian kanan
3  let L[1..n1 + 1] and R[1..n2 + 1] be new arrays
4  for i = 1 to n1
5      L[i] = A[p + i - 1]           // salin bagian kiri ke L
6  for j = 1 to n2
7      R[j] = A[q + j]               // salin bagian kanan ke R
8  L[n1 + 1] = ∞                     // sentinel (penjaga)
9  R[n2 + 1] = ∞
10 i = 1
11 j = 1
12 for k = p to r                    // isi kembali A[p..r] satu per satu
13     if L[i] ≤ R[j]
14         A[k] = L[i]
15         i = i + 1
16     else A[k] = R[j]
17         j = j + 1
```

- **Baris 1–7**: menyalin bagian kiri ke array L dan bagian kanan ke array R.
- **Baris 8–9**: menaruh nilai ∞ di ujung L dan R sebagai **sentinel** (penjaga). Ketika salah satu array sudah "habis", ujungnya berisi ∞ sehingga perbandingan pada baris 13 otomatis selalu memilih elemen dari array yang lain. Dengan sentinel, kita tidak perlu memeriksa secara khusus apakah salah satu array sudah kosong.
- **Baris 12–17**: untuk setiap posisi k dari p sampai r, bandingkan elemen terdepan $L[i]$ dan $R[j]$, salin yang lebih kecil ke $A[k]$, lalu majukan penunjuk array asalnya.

Perulangan pada baris 12 berjalan tepat $r - p + 1 = n$ kali, dan setiap putarannya berbiaya konstan. Penyalinan pada baris 4–7 juga sebanyak n elemen. Jadi **MERGE berjalan dalam waktu** $\Theta(n)$, dengan n adalah banyaknya elemen yang digabung.

3.3 Contoh satu kali MERGE

Misalkan $A[1..5] = (3, 8, 10, 5, 20)$ dengan $p = 1$, $q = 3$, $r = 5$. Bagian kiri $(3, 8, 10)$ dan bagian kanan $(5, 20)$ sama-sama sudah terurut. Maka $L = (3, 8, 10, \infty)$ dan $R = (5, 20, \infty)$.

k	$L[i]$	$R[j]$	Pilih	$A[1..k]$ setelah langkah
1	3	5	L: 3	3
2	8	5	R: 5	3, 5
3	8	20	L: 8	3, 5, 8
4	10	20	L: 10	3, 5, 8, 10
5	∞	20	R: 20	3, 5, 8, 10, 20

Tabel 2. Penelusuran MERGE(A, 1, 3, 5).

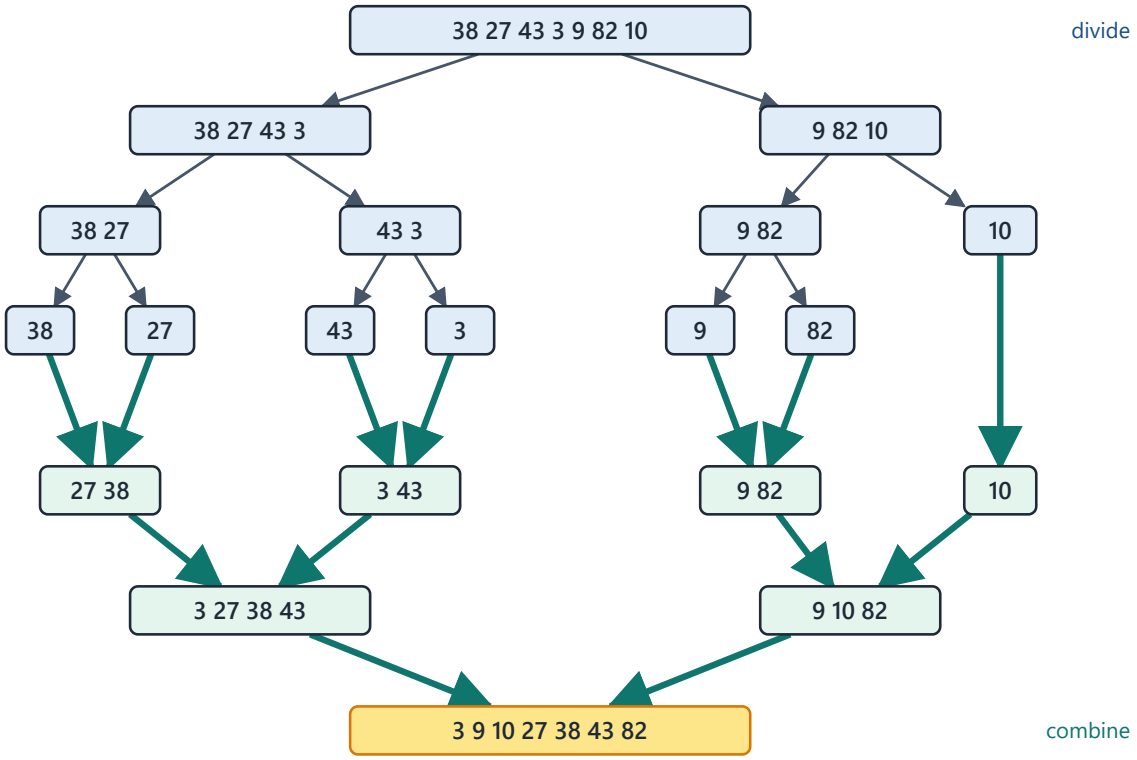
Pada $k = 5$, L sudah habis sehingga $L[i] = \infty$, dan sentinel membuat $R[j] = 20$ terpilih tanpa pemeriksaan khusus.

CONTOH DAN PENERAPAN

4. Penelusuran Merge Sort

4.1 Contoh 1: array berisi 7 bilangan

Contoh dari bahan kuliah: $A = (38, 27, 43, 3, 9, 82, 10)$. Awalnya $p = 1$, $r = 7$, sehingga $q = \lfloor 8/2 \rfloor = 4$. Gambar berikut memperlihatkan seluruh proses: bagian atas adalah tahap *divide* (pemanggilan rekursif terus membagi sampai satu elemen), bagian bawah adalah tahap *combine* (MERGE menyatukan kembali).



MERGE terakhir: L = (3, 27, 38, 43, ∞), R = (9, 10, 82, ∞)

Gambar 1. Merge Sort pada (38, 27, 43, 3, 9, 82, 10). Kotak biru: hasil pembagian; kotak hijau: hasil penggabungan.

Perhatikan bahwa elemen 10 tidak punya pasangan pada tingkat ketiga karena banyaknya elemen ganjil. Ia langsung menjadi kasus dasar `MERGE_SORT(A, 7, 7)`, lalu ikut digabung pada tingkat berikutnya.

4.2 Contoh 2: urutan pemanggilan untuk {3, 10, 8, 20, 5}

Soal dari bahan kuliah: tuliskan langkah-langkah Merge Sort untuk mengurutkan $\{3, 10, 8, 20, 5\}$. Bahan kuliah menelusuri sampai terbentuk $(3, 8, 10)$ dan meninggalkan sisanya sebagai latihan. Berikut penelusuran **lengkap**. Indentasi menunjukkan kedalaman rekursi; baris yang sama tingkat indentasinya berasal dari pemanggilan yang sama.

Penelusuran lengkap MERGE_SORT(A, 1, 5) untuk A = (3, 10, 8, 20, 5)

```
MERGE_SORT(A,1,5): 1 < 5 TRUE, q = ⌊6/2⌋ = 3
  MERGE_SORT(A,1,3): 1 < 3 TRUE, q = ⌊4/2⌋ = 2
    MERGE_SORT(A,1,2): 1 < 2 TRUE, q = ⌊3/2⌋ = 1
      MERGE_SORT(A,1,1): 1 < 1 FALSE → kembali
      MERGE_SORT(A,2,2): 2 < 2 FALSE → kembali
      MERGE(A,1,1,2): (3) + (10)          → A[1..2] = (3, 10)
    MERGE_SORT(A,3,3): 3 < 3 FALSE → kembali
    MERGE(A,1,2,3): (3, 10) + (8)          → A[1..3] = (3, 8, 10)
  ----------------- bagian di bawah ini adalah lanjutan latihan -----------------
  MERGE_SORT(A,4,5): 4 < 5 TRUE, q = ⌊9/2⌋ = 4
    MERGE_SORT(A,4,4): 4 < 4 FALSE → kembali
    MERGE_SORT(A,5,5): 5 < 5 FALSE → kembali
    MERGE(A,4,4,5): (20) + (5)             → A[4..5] = (5, 20)
  MERGE(A,1,3,5): (3, 8, 10) + (5, 20)     → A[1..5] = (3, 5, 8, 10, 20)
```

MERGE terakhir, `MERGE(A,1,3,5)`, persis contoh satu kali MERGE yang ditelusuri di atas. Hasil akhirnya $(3, 5, 8, 10, 20)$, sesuai jawaban yang diharapkan bahan kuliah.

Ada pola penting yang terlihat dari penelusuran ini: Merge Sort menyelesaikan **seluruh** bagian kiri lebih dulu (sampai $A[1..3]$ terurut) sebelum mulai mengerjakan bagian kanan. Ini akibat langsung dari cara kerja call stack (Modul 02): pemanggilan pada baris 4 baru dimulai setelah pemanggilan pada baris 3 selesai sepenuhnya.

4.3 Implementasi Python

Python: Merge Sort dengan jejak pemanggilan MERGE

```python
INF = float("inf")

def merge(A, p, q, r):
    L = A[p:q + 1] + [INF]           # A[p..q] ditambah sentinel
    R = A[q + 1:r + 1] + [INF]       # A[q+1..r] ditambah sentinel
    i = j = 0
    for k in range(p, r + 1):
        if L[i] <= R[j]:
            A[k] = L[i]; i += 1
        else:
            A[k] = R[j]; j += 1

def merge_sort(A, p, r):
    if p < r:
        q = (p + r) // 2
        merge_sort(A, p, q)
        merge_sort(A, q + 1, r)
        merge(A, p, q, r)
        print(f"MERGE({p},{q},{r}) -> {A[p:r + 1]}")

A = [None, 3, 10, 8, 20, 5]          # A[1..5], indeks 0 tidak dipakai
merge_sort(A, 1, 5)
print(A[1:])
```

Keluaran program

```
MERGE(1,1,2) -> [3, 10]
MERGE(1,2,3) -> [3, 8, 10]
MERGE(4,4,5) -> [5, 20]
MERGE(1,3,5) -> [3, 5, 8, 10, 20]
[3, 5, 8, 10, 20]
```

PEMBAHASAN INTI

5. Analisis Running Time: Lahirnya Recurrence

Misalkan $T(n)$ adalah running time Merge Sort untuk n bilangan. Kita uraikan biaya setiap tahap:

- **Divide**: hanya menghitung indeks tengah, sehingga $D(n) = \Theta(1)$.
- **Conquer**: menyelesaikan dua submasalah yang masing-masing berukuran $n/2$, sehingga biayanya $2T(n/2)$.
- **Combine**: MERGE pada n elemen, sehingga $C(n) = \Theta(n)$.

Menjumlahkan ketiganya:

$$T(n) = D(n) + 2T(n/2) + C(n) = \Theta(1) + 2T(n/2) + \Theta(n) = 2T(n/2) + \Theta(n), \quad n > 1.$$

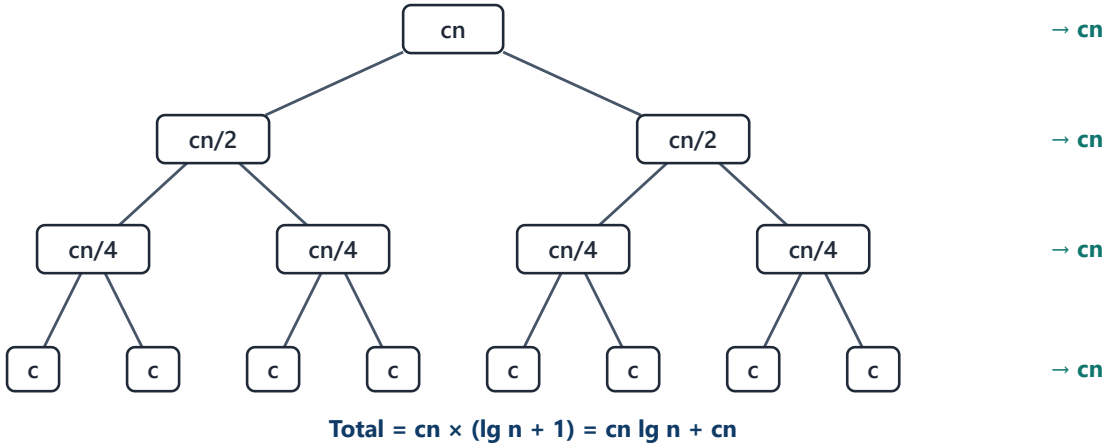
(Suku $\Theta(1)$ terserap ke dalam $\Theta(n)$ karena $\Theta(n) + \Theta(1) = \Theta(n)$.) Untuk $n = 1$, Merge Sort hanya memeriksa kondisi $p < r$, sehingga $T(1) = \Theta(1)$.

DEFINISI

Persamaan seperti $T(n) = 2T(n/2) + \Theta(n)$ disebut **recurrence**: fungsi running time yang di dalamnya mengandung fungsi running time itu sendiri untuk ukuran yang lebih kecil. Recurrence adalah cara alami menyatakan running time algoritma rekursif.

5.1 Intuisi penyelesaian: pohon rekursi

Ganti $\Theta(n)$ dengan cn untuk suatu konstanta c, lalu gambarkan biaya setiap pemanggilan sebagai pohon. Akar berbiaya cn (MERGE pada n elemen). Dua anaknya masing-masing berbiaya $c\frac{n}{2}$, empat cucunya masing-masing $c\frac{n}{4}$, dan seterusnya, sampai daun berukuran 1.



Gambar 2. Pohon rekursi untuk $T(n) = 2T(n/2) + cn$ (digambar untuk n = 8). Setiap tingkat berbiaya total cn, dan ada lg n + 1 tingkat.

Setiap tingkat berbiaya total cn: tingkat ke-i memiliki 2^i simpul yang masing-masing berbiaya $c\frac{n}{2^i}$. Ukuran submasalah dibagi dua di setiap tingkat, sehingga dari n sampai 1 dibutuhkan $\lg n$ kali pembagian, jadi ada $\lg n + 1$ tingkat. Total biaya:

$$T(n) = cn(\lg n + 1) = cn \lg n + cn = \Theta(n \lg n).$$

Penjelasan ini masih berupa intuisi (misalnya, kita mengandaikan n adalah pangkat dari 2). Modul 09 menyajikan cara yang lebih sistematis untuk menyelesaikan recurrence semacam ini, yaitu **Metode Master**, yang memberikan hasil yang sama.

ANALISIS DAN HUBUNGAN KONSEP

6. Merge Sort dibandingkan dengan Insertion Sort

Aspek	Insertion Sort (Modul 04)	Merge Sort
Paradigma	Inkremental	Divide and Conquer
Kasus terbaik	$\Theta(n)$ (data terurut)	$\Theta(n \lg n)$
Kasus terburuk	$\Theta(n^2)$	$\Theta(n \lg n)$
Memori tambahan	Konstan (di tempat)	$\Theta(n)$ untuk array L dan R
Stabil?	Ya	Ya (karena baris 13 memakai $\leq$)
Cocok untuk	Data kecil atau hampir terurut	Data besar, kinerja yang dapat diprediksi

Tabel 3. Perbandingan Insertion Sort dan Merge Sort.

Running time yang tidak bergantung pada isi data. Berbeda dengan Insertion Sort, Merge Sort selalu membagi dan menggabungkan dengan cara yang sama apa pun isi datanya. Itulah sebabnya kasus terbaik dan terburuknya sama-sama $\Theta(n \lg n)$.

Mengapa lebih cepat? Insertion Sort memperbaiki urutan dengan menggeser elemen satu posisi demi satu posisi. Merge Sort justru menghindari pekerjaan itu: setiap elemen hanya "disentuh" sekali di setiap tingkat pohon rekursi, dan pohonnya hanya setinggi $\lg n + 1$ tingkat. Berdasarkan Modul 07, $n \lg n = o(n^2)$

, sehingga untuk data yang cukup besar Merge Sort pasti lebih cepat. Modul 05 memperkirakan titik potongnya terjadi di sekitar $n = 190$ untuk konstanta ilustratif $2n^2$ vs $50\, n \lg n$.

Hubungan dengan modul lain. Recurrence $T(n) = 2T(n/2) + \Theta(n)$ diselesaikan secara formal di Modul 09. Paradigma Divide and Conquer juga menjadi pembanding penting untuk **Dynamic Programming** (Modul 10): keduanya memecah masalah menjadi submasalah, tetapi Dynamic Programming dipakai ketika submasalahnya saling tumpang tindih, sesuatu yang tidak terjadi pada Merge Sort.

KESIMPULAN

7. Kesimpulan

- Divide and Conquer terdiri atas tiga tahap: divide (bagi masalah), conquer (selesaikan submasalah secara rekursif), dan combine (gabungkan solusi).
- Merge Sort membagi array menjadi dua bagian di indeks tengah $q = \lfloor (p + r)/2 \rfloor$, mengurutkan masing-masing secara rekursif, lalu menggabungkannya dengan MERGE. Rekursi berhenti ketika $p \geq r$.
- MERGE menggabungkan dua subarray terurut dalam waktu $\Theta(n)$; sentinel ∞ menyederhanakan penanganan subarray yang sudah habis.
- Running time Merge Sort memenuhi recurrence $T(n) = 2T(n/2) + \Theta(n)$, yang solusinya $\Theta(n \lg n)$ untuk semua masukan.
- Merge Sort jauh lebih cepat daripada Insertion Sort untuk data besar, dengan biaya memori tambahan $\Theta(n)$.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 **PILIHAN GANDA**

Pada Merge Sort, tahap manakah yang benar-benar melakukan pekerjaan "mengurutkan" elemen?

A. Divide, karena membagi array di tengah.
B. Combine (MERGE), karena di sinilah elemen dibandingkan dan disusun berurutan.
C. Kasus dasar, karena subarray satu elemen sudah terurut.
D. Tidak ada tahap yang mengurutkan.

2 **PENELUSURAN**

Tuliskan urutan pemanggilan `MERGE` (dengan parameter p, q, r) yang terjadi saat `MERGE_SORT(A, 1, 6)` dijalankan pada $A = (5, 2, 4, 6, 1, 3)$, beserta hasil setiap MERGE.

3 **PENELUSURAN**

Telusuri `MERGE(A, 1, 2, 4)` untuk $A = (4, 9, 1, 7)$. Tuliskan isi L dan R (termasuk sentinel) serta elemen yang dipilih pada setiap k.

4 **BENAR/SALAH DENGAN ALASAN**

"Jika array sudah terurut, Merge Sort berjalan dalam waktu $\Theta(n)$ seperti Insertion Sort." Benar atau salah?

5 **ANALISIS**

Apa fungsi sentinel ∞ pada MERGE? Apa yang harus ditambahkan ke pseudocode jika sentinel tidak dipakai?

6 **PERHITUNGAN**

Untuk $n = 16$, berapa banyak tingkat pada pohon rekursi Merge Sort, dan berapa total biaya bila setiap tingkat berbiaya cn? Bandingkan dengan cn^2.

7 **ANALISIS KASUS**

Sebuah algoritma Divide and Conquer membagi masalah berukuran n menjadi **tiga** submasalah berukuran $n/3$, dan menggabungkan hasilnya dalam waktu $\Theta(n)$. Tuliskan recurrence running time-nya.

8 **PILIHAN GANDA**

Jika baris 13 MERGE diubah dari `L[i] ≤ R[j]` menjadi `L[i] < R[j]`, apa akibatnya?

A. Hasil tidak lagi terurut.
B. Hasil tetap terurut, tetapi Merge Sort tidak lagi stabil.
C. Running time menjadi $\Theta(n^2)$.
D. Algoritma tidak berhenti.

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

Jawaban: B. Tahap divide hanya menghitung indeks tengah tanpa membandingkan isi array. Seluruh perbandingan dan penyusunan terjadi di MERGE.

Soal 2

Pembagian: $(1, 6)$ dengan $q = 3$; kiri $(1, 3)$ dengan $q = 2$; kanan $(4, 6)$ dengan $q = 5$. Urutan MERGE:

1. `MERGE(1,1,2)` : $(5) + (2) \to (2, 5)$
2. `MERGE(1,2,3)` : $(2, 5) + (4) \to (2, 4, 5)$
3. `MERGE(4,4,5)` : $(6) + (1) \to (1, 6)$
4. `MERGE(4,5,6)` : $(1, 6) + (3) \to (1, 3, 6)$
5. `MERGE(1,3,6)` : $(2, 4, 5) + (1, 3, 6) \to (1, 2, 3, 4, 5, 6)$

Soal 3

$L = (4, 9, \infty)$, $R = (1, 7, \infty)$. $k = 1$: 4 vs 1, pilih R: 1. $k = 2$: 4 vs 7, pilih L: 4. $k = 3$: 9 vs 7, pilih R: 7. $k = 4$: 9 vs ∞, pilih L: 9. Hasil $A = (1, 4, 7, 9)$.

Soal 4

Salah. Merge Sort tetap membagi sampai satu elemen dan tetap menjalankan setiap MERGE yang masing-masing menyalin dan memeriksa semua elemen, apa pun isinya. Running time-nya tetap $\Theta(n \lg n)$.

Soal 5

Sentinel memastikan bahwa ketika salah satu dari L atau R sudah habis, perbandingan $L[i] \le R[j]$ otomatis memilih elemen dari array yang masih tersisa. Tanpa sentinel, setiap putaran harus memeriksa dulu apakah $i > n_1$ atau $j > n_2$, atau setelah salah satu habis, sisa array yang lain disalin langsung ke A.

Soal 6

Tingkatnya $\lg 16 + 1 = 5$. Total biaya $= 5 \cdot 16c = 80c$. Sebagai pembanding, $cn^2 = 256c$. Untuk n yang lebih besar, misalnya $n = 1024$, perbedaannya jauh lebih mencolok: $11 \cdot 1024c = 11.264c$ vs $1.048.576c$.

Soal 7

$T(n) = 3T(n/3) + \Theta(n)$ untuk $n > 1$, dengan $T(1) = \Theta(1)$. (Dengan Metode Master di Modul 09, hasilnya juga $\Theta(n \lg n)$.)

Soal 8

Jawaban: B. Ketika $L[i] = R[j]$, versi baru mengambil elemen dari R lebih dulu, sehingga elemen bernilai sama yang semula berada di bagian kiri ditempatkan setelah elemen dari bagian kanan. Urutan nilai tetap benar, tetapi urutan relatif elemen yang sama berubah.

Referensi

Sumber utama: Materi 6. Algoritma Divide and Conquer (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Knuth, D. E. (1997). *The Art of Computer Programming, Vol. 1: Fundamental Algorithms* (3rd ed.). Reading, MA: Addison-Wesley. (Bagian 1.1 tentang lima sifat algoritma.)
3. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Upper Saddle River, NJ: Addison-Wesley.